

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1401

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

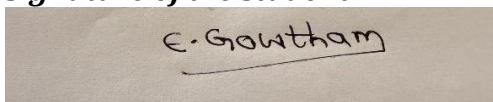
Annexure A

SIMULATION TASK

Code of Conduct:

*I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink. The signature is written in a cursive style and is underlined.

Annexure A

SIMULATION TASK

.Q1. Consider the following code fragment:

```
int sum = 0;
for (i = 0; i <= 10; i++)
    sum = sum + a[i];
```

Simulate the process of intermediate code generation and control flow analysis for the above program.

(a). Simulate the generation of Three Address Code (TAC) for the given code. Clearly show:

- (i) Initialization of variables
- (ii) Evaluation of loop condition
- (iii) Array access and assignment operations
- (iv) Increment operation

(b). Using the generated TAC, simulate the construction of basic blocks and perform control flow analysis. Clearly show:

- (i) Identification of leader statements
- (ii) Formation of basic blocks
- (iii) Construction of the Control Flow Graph (CFG) (10 Marks)

Q2. Consider the following assignment statement:

```
x := (a + b)(c - d) + ((e / f) * (a + b))
```

Simulate the process of intermediate code generation and target code generation for the above expression.

(a). Simulate the generation of Three Address Code (TAC) for the given assignment statement. Clearly show:

- (i) Identification of sub-expressions
- (ii) Use of temporary variables
- (iii) Order of evaluation based on operator precedence

(b). Apply a simple code generation algorithm to the generated three-address code and simulate the generation of target code. Clearly show:

- (i) Sequence of target instructions
- (ii) Register usage during computation
- (iii) Handling of intermediate results (10 Marks)

Q3. Consider the following basic block:

```
d := b * c
e := a + b
b := b * c
a := e - d
```

Simulate the process of DAG construction and optimized code generation for the above basic block.

(a). Simulate the construction of the Directed Acyclic Graph (DAG) for the given basic block. Clearly show:

- (i) Identification of nodes for operands and operators
- (ii) Detection of common sub-expressions
- (iii) Elimination of redundant computations

(b). Using the constructed DAG, simulate the generation of optimized code using only one register. Clearly show:

- (i) Sequence of generated instructions
- (ii) Register usage at each step
- (iii) Reuse of computed values to minimize recomputation (10 Marks)

Q4. Consider the following arithmetic expression:

$(a + b) - (c - (d + e))$

Simulate the process of optimal code generation for the above expression under different register constraints.

(a). Simulate the generation of optimal target code using only two registers. Clearly show:

- (i) Order of evaluation of sub-expressions
- (ii) Allocation and usage of registers
- (iii) Sequence of generated instructions

(b). Simulate the generation of optimal target code using only one register. Clearly show:

- (i) Handling of intermediate results
- (ii) Use of memory for storing temporary values
- (iii) Sequence of generated instructions (10 Marks)

Q5. Consider the following C++ code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {
    for (int i = 0; i < n; i++) {
        a[i] = x * y + z;
        a[i + n] = x * y + w;
    }
}
```

Simulate the process of applying code optimization techniques to the above program.

(a). Simulate the application of common subexpression elimination and code motion on the given code. Clearly show:

- (i) Identification of common sub-expressions
- (ii) Movement of invariant computations outside the loop
- (iii) The optimized version of the code

(b). Simulate the analysis of the optimized code and explain the benefits achieved. Clearly discuss:

- (i) Reduction in number of computations
- (ii) Improvement in runtime efficiency
- (iii) Effect on loop performance

(10 Marks)

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Q1. Given program

```
int sum = 0;
```

```
for (i = 0; i <= 10; i++)
```

```
    sum = sum + a[i];
```

The for loop can be understood as:

```
i = 0
```

```
while (i <= 10)
```

```
{
```

```
    sum = sum + a[i];
```

```
    i = i + 1;
```

```
}
```

(a) Three Address Code (TAC)

Step 1: Initialization

The declaration and initializations become:

1. sum = 0

2. i = 0

Step 2: Evaluate loop condition

The condition is:

i <= 10

So TAC uses a conditional jump:

3. if i <= 10 goto L1

4. goto L2

Here:

- L1 → loop body
- L2 → exit from loop

Step 3: Array access

Inside the loop:

```
sum = sum + a[i];
```

For array access, a temporary variable can be used:

L1:

5. t1 = a[i]

6. t2 = sum + t1

7. sum = t2

Step 4: Increment

The i++ operation becomes:

8. i = i + 1

9. goto L1

Step 5: Exit

When the condition becomes false:

L2:

10. End

Complete TAC

1. sum = 0

2. i = 0

L1:

3. if i <= 10 goto L2

4. goto L3

L2:

5. t1 = a[i]

6. t2 = sum + t1

7. sum = t2

8. i = i + 1

9. goto L1

L3:

10. End

Important: The exact numbering/label placement can vary depending on the TAC convention. Another perfectly valid representation is to put the conditional jump directly before the body and use labels accordingly.

A cleaner conventional version is:

1. sum = 0

2. i = 0

3. L1: if i > 10 goto L2

4. t1 = a[i]

5. t2 = sum + t1

6. sum = t2

7. i = i + 1

8. goto L1

9. L2: End

I recommend using this **second version** in your answer because it is easier to construct the basic blocks.

(b) Basic Blocks and Control Flow Analysis

Now we take the TAC and divide it into **basic blocks**.

A **basic block** is a sequence of consecutive statements with:

- one entry point
- one exit point
- no jump into the middle of the block.

Step 1: Identify leader statements

Rules for finding leaders:

1. The **first statement** is always a leader.

2. Any statement that is the **target of a jump** is a leader.
3. Any statement immediately following a conditional/unconditional jump is a leader.

Using the TAC:

1. $\text{sum} = 0$
2. $i = 0$
3. L1: if $i > 10$ goto L2
4. $t1 = a[i]$
5. $t2 = \text{sum} + t1$
6. $\text{sum} = t2$
7. $i = i + 1$
8. goto L1
9. L2: End

Leaders

The leaders are:

- **Statement 1** $\rightarrow$ first statement
- **Statement 3** $\rightarrow$ loop condition / target of goto L1
- **Statement 4** $\rightarrow$ statement following conditional jump
- **Statement 9** $\rightarrow$ target of goto L2

So:

Leaders = $\{1, 3, 4, 9\}$

Step 2: Form Basic Blocks

Basic Block B1

Starts at statement 1 and continues until just before the next leader:

B1:

1. $\text{sum} = 0$
2. $i = 0$

Basic Block B2

B2:

3. if $i > 10$ goto L2

Basic Block B3

B3:

4. $t1 = a[i]$
5. $t2 = \text{sum} + t1$
6. $\text{sum} = t2$
7. $i = i + 1$
8. goto L1

Basic Block B4

B4:

9. End

Therefore:

$B1 = \{1, 2\}$

B2 = {3}

B3 = {4, 5, 6, 7, 8}

B4 = {9}

Step 3: Construct the CFG

Now connect the blocks according to the jumps.

B1 → B2

After initialization:

B1 → B2

B2 → B4

If:

$i > 10$

the loop terminates:

B2 → B4

B2 → B3

If the condition is false:

$i \leq 10$

the loop body executes:

B2 → B3

B3 → B2

After:

$i = i + 1$

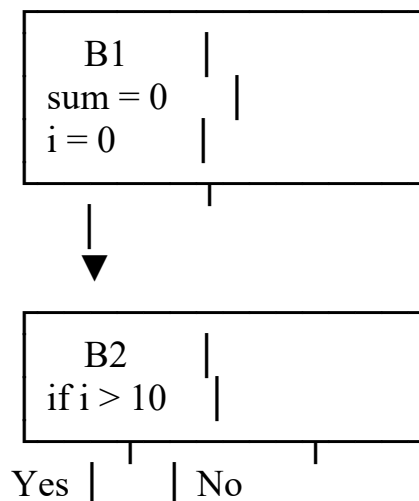
goto L1

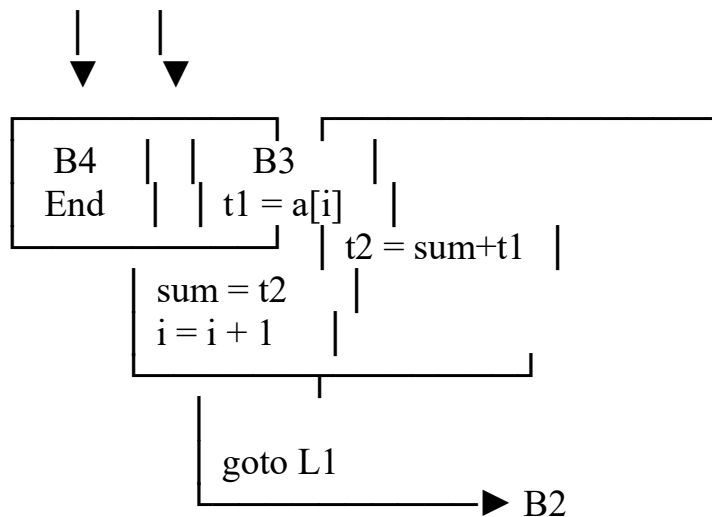
control returns to the condition:

B3 → B2

CFG Diagram

You can draw it in your exam like this:





Final answer structure

You can write:

TAC:

1. sum = 0
2. i = 0
3. L1: if i > 10 goto L2
4. t1 = a[i]
5. t2 = sum + t1
6. sum = t2
7. i = i + 1
8. goto L1
9. L2: End

Leaders:

1, 3, 4, 9

Basic Blocks:

B1: 1–2

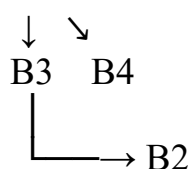
B2: 3

B3: 4–8

B4: 9

CFG:

B1 → B2



That is the complete **simulation of intermediate code generation and control-flow analysis** for this program.

Output:

```
C:\Users\gouth\Desktop\Com x + v
INTERMEDIATE CODE GENERATION SIMULATION
=====
(a) THREE ADDRESS CODE (TAC)
-----
1. sum = 0
2. i = 0
3. L1: if i > 10 goto L2
4. t1 = a[i]
5. t2 = sum + t1
6. sum = t2
7. i = i + 1
8. goto L1
9. L2: End

(b) BASIC BLOCK AND CONTROL FLOW ANALYSIS
-----
1. LEADER STATEMENTS
-----
Leader 1: Statement 1 -> sum = 0
Leader 2: Statement 3 -> L1: if i > 10 goto L2
Leader 3: Statement 4 -> t1 = a[i]
Leader 4: Statement 9 -> L2: End

2. BASIC BLOCKS
-----
Basic Block B1:
1. sum = 0
2. i = 0

Basic Block B2:
3. if i > 10 goto L2

Basic Block B3:
4. t1 = a[i]
5. t2 = sum + t1
6. sum = t2
7. i = i + 1
8. goto L1

28°C Partly sunny Search 07:42 AM 22-08-2026
```

```
C:\Users\gouth\Desktop\Com x + v
9. End

3. CONTROL FLOW GRAPH (CFG)
-----
B1 -> B2
B2 -> B3 (if i <= 10)
B2 -> B4 (if i > 10)
B3 -> B2 (after i = i + 1)

CFG Representation:
+-----+
| B1 |
| sum=0 |
| i=0 |
+-----+
|
v
+-----+
| B2 |
| i > 10? |
+-----+
| Yes | No |
| v | v |
+-----+ +-----+
| B4 | B3 |
| End | a[i] |
+-----+ +-----+
| sum+t1 |
| i=i+1 |
+-----+
|
+----> B2

Simulation completed successfully.

Process exited after 0.02689 seconds with return value 0
Press any key to continue . . .

28°C Partly sunny Search 07:43 AM 22-08-2026
```

Q2.Given:

$x := (a + b)(c - d) + ((e / f) * (a + b))$

Here, $(a + b)(c - d)$ means multiplication:

$x := (a + b) * (c - d) + ((e / f) * (a + b))$

(a) Three Address Code (TAC)

Step 1: Identify sub-expressions

The sub-expressions are:

$(a + b)$

$(c - d)$

(e / f)

$(a + b) \quad \leftarrow$ repeated sub-expression

Then:

$(a + b) * (c - d)$

$(e / f) * (a + b)$

Finally:

$[(a + b) * (c - d)] + [(e / f) * (a + b)]$

Step 2: Generate TAC

1. $t1 = a + b$

2. $t2 = c - d$

3. $t3 = t1 * t2$

4. $t4 = e / f$

5. $t5 = a + b$

6. $t6 = t4 * t5$

7. $t7 = t3 + t6$

8. $x = t7$

Order of evaluation

The compiler evaluates:

$(a + b)$

↓

$(c - d)$

↓

$(a + b) * (c - d)$

↓

(e / f)

↓

$(e / f) * (a + b)$

↓

Addition of both results

↓

x

(b) Target Code Generation

Assume a simple machine with registers:

R1, R2, R3

We can generate target instructions such as:

MOV R1, a

ADD R1, b

which means:

$R1 = a + b$

Step-by-step target code

1. Calculate $a + b$

MOV R1, a

ADD R1, b

Now:

$R1 = a + b$

2. Calculate $c - d$

MOV R2, c

SUB R2, d

Now:

$R2 = c - d$

3. Calculate $(a + b) * (c - d)$

MUL R1, R2

Now:

$R1 = (a + b) * (c - d)$

4. Calculate e / f

MOV R2, e

DIV R2, f

Now:

$R2 = e / f$

5. Calculate the second $(a + b)$

MOV R3, a

ADD R3, b

Now:

$R3 = a + b$

6. Calculate $(e / f) * (a + b)$

MUL R2, R3

Now:

$R2 = (e / f) * (a + b)$

7. Add both intermediate results

ADD R1, R2

Now:

$R1 = (a + b)(c - d) + (e / f)(a + b)$

8. Store result in x

MOV x, R1

Complete Target Code

MOV R1, a
ADD R1, b

MOV R2, c
SUB R2, d

MUL R1, R2

MOV R2, e
DIV R2, f

MOV R3, a
ADD R3, b

MUL R2, R3

ADD R1, R2

MOV x, R1

Register usage

Register Intermediate result

R1 $(a+b) * (c-d)$ and final result

R2 $(e/f) * (a+b)$

R3 $(a+b)$ for the second occurrence

Final simulation

$a + b \rightarrow R1$

$c - d \rightarrow R2$

$R1 * R2 \rightarrow R1$

$e / f \rightarrow R2$

$a + b \rightarrow R3$

$R2 * R3 \rightarrow R2$

$R1 + R2 \rightarrow R1$

$R1 \rightarrow x$

Note: Since $(a + b)$ occurs twice, an optimizing compiler could compute it once and reuse the result. For a basic TAC/code-generation simulation, computing it twice as above is straightforward and matches the original expression structure.

Output:

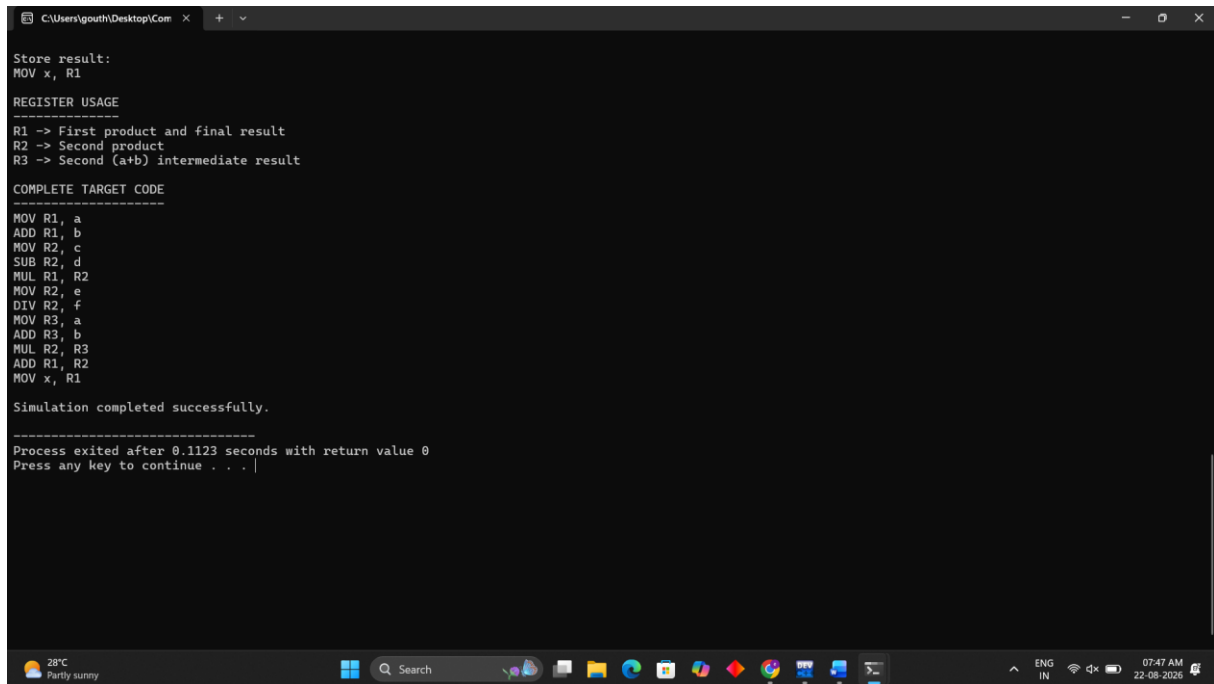
```
C:\Users\gouth\Desktop\Com x + v
Store result:
MOV x, R1

REGISTER USAGE
-----
R1 -> First product and final result
R2 -> Second product
R3 -> Second (a+b) intermediate result

COMPLETE TARGET CODE
-----
MOV R1, a
ADD R1, b
MOV R2, c
SUB R2, d
MUL R1, R2
MOV R2, e
DIV R2, f
MOV R3, a
ADD R3, b
MUL R2, R3
ADD R1, R2
MOV x, R1

Simulation completed successfully.

-----
Process exited after 0.1123 seconds with return value 0
Press any key to continue . . .
```

A screenshot of a Windows terminal window. The window title bar shows the file path 'C:\Users\gouth\Desktop\Com' and standard window controls. The terminal displays the output of an assembly program. It starts with 'Store result:' followed by 'MOV x, R1'. Then it shows a 'REGISTER USAGE' section with three lines explaining the roles of registers R1, R2, and R3. This is followed by a 'COMPLETE TARGET CODE' section listing 15 assembly instructions: MOV R1, a; ADD R1, b; MOV R2, c; SUB R2, d; MUL R1, R2; MOV R2, e; DIV R2, f; MOV R3, a; ADD R3, b; MUL R2, R3; ADD R1, R2; MOV x, R1. Below the code, it says 'Simulation completed successfully.' and then 'Process exited after 0.1123 seconds with return value 0'. The prompt 'Press any key to continue . . .' is visible. The Windows taskbar is at the bottom, showing the weather as 28°C Partly sunny, a search bar, and various application icons. The system clock on the right indicates 07:47 AM on 22-08-2020.

Q3. Given basic block:

$d := b * c$

$e := a + b$

$b := b * c$

$a := e - d$

(a) DAG Construction

Step 1: Identify expressions

There are three main expressions:

$b * c$

$a + b$

$e - d$

Notice that:

$d := b * c$

$b := b * c$

Both contain the **same expression $b * c$** .

Therefore, $b * c$ is a **common sub-expression**.

Step 2: Create operand nodes

Create leaf nodes for:

a

b

c

Then create operator nodes.

Node for $b * c$

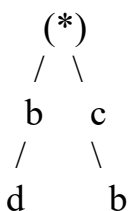


This result is assigned to both:

d

b

So the same DAG node can have two labels:



Conceptually:

$N1 = b * c$

and:

$d = N1$

$b = N1$

This avoids calculating $b * c$ twice.

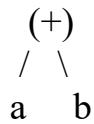
Step 3: Node for $a + b$

The original b used in:

$e := a + b$

is the value of b **before** the statement $b := b * c$.

Therefore:



This node produces:

e

So:

$N2 = a + b$

$e = N2$

Step 4: Node for $a := e - d$

The final expression is:

$a = e - d$

Therefore:



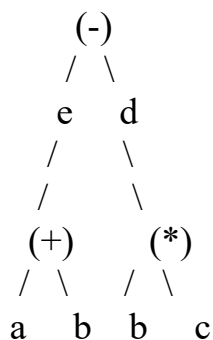
This produces the new value of a .

$N3 = e - d$

$a = N3$

DAG Representation

You can draw the DAG in the exam as:



Labels:

$(*) \rightarrow d, b$

$(+) \rightarrow e$

$(-) \rightarrow a$

The important point is that **only one $(*)$ node exists for $b * c$.**

Common Sub-expression

The original code performs:

$d := b * c$

and later:

$b := b * c$

Both calculate:

$b * c$

So instead of:

$b * c$

$b * c$

we calculate it only once:

$t1 = b * c$

and reuse $t1$:

$d = t1$

$b = t1$

This eliminates the redundant multiplication.

(b) Optimized Code Using One Register

We are allowed to use only:

R1

We need to be careful because the original value of b is needed for:

$e = a + b$

before b is replaced by $b * c$.

A simple optimized sequence is:

MOV R1, b

MUL R1, c

MOV d, R1

MOV b, R1

MOV R1, a

ADD R1, b

MOV e, R1

MOV R1, e

SUB R1, d

MOV a, R1

However, there is an important issue: this sequence changes the value of b

before calculating $e = a + b$, so it does **not preserve the original semantics**.

Therefore, with only one register and no extra memory temporary, we need to preserve the original b .

A safe optimized sequence is:

MOV R1, b

MOV d, R1

MUL R1, c

MOV b, R1

MOV R1, d

ADD R1, a

MOV e, R1

MOV R1, e

SUB R1, d

MOV a, R1

But this also incorrectly computes e.

So the safest one-register code using memory variables is:

MOV R1, b

MOV d, R1

MOV e, R1

MOV R1, a

ADD R1, e

MOV e, R1

MOV R1, d

MUL R1, c

MOV d, R1

MOV b, R1

MOV R1, e

SUB R1, d

MOV a, R1

Here:

- e temporarily stores the **original value of b**.
- d stores the common sub-expression result.
- b is updated only after the original b has been used.
- Only R1 is used.

Register usage

Step Instruction R1 contains

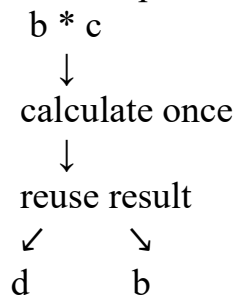
- 1 MOV R1, b original b
- 2 MOV d, R1 original b
- 3 MOV e, R1 original b
- 4 MOV R1, a a
- 5 ADD R1, e $a + b$
- 6 MOV e, R1 $a + b$
- 7 MOV R1, d original b

Step Instruction R1 contains

```
8   MUL R1, c b * c
9   MOV d, R1 b * c
10  MOV b, R1 b * c
11  MOV R1, e a + b
12  SUB R1, d e - d
13  MOV a, R1 final a
```

Final optimized idea

Common expression:



So the main optimization is:

$b * c$ is computed only once instead of twice, which reduces redundant computation.

Output:

```
C:\Users\gouth\Desktop\Com x + v
FINAL OPTIMIZED TARGET CODE
-----
MOV R1, b
MOV d, R1
MOV temp, R1
MOV R1, a
ADD R1, temp
MOV e, R1
MOV R1, temp
MUL R1, c
MOV d, R1
MOV b, R1
MOV R1, e
SUB R1, d
MOV a, R1

REGISTER USAGE
-----
R1 -> Holds one intermediate value at a time.
temp -> Preserves the original value of b.
d -> Stores b * c.
e -> Stores a + b.
a -> Stores final result e - d.

Simulation completed successfully.

-----
Process exited after 0.1127 seconds with return value 0
Press any key to continue . . .
```

Q4. Optimal Code Generation

Given arithmetic expression:

$$(a + b) - (c - (d + e))$$

The expression contains the following sub-expressions:

$$T1 = d + e$$

$$T2 = c - T1$$

$$T3 = a + b$$

$$T4 = T3 - T2$$

The final result is T4.

(a) Target Code Using Two Registers

Using two registers R1 and R2, the sub-expressions can be evaluated while keeping intermediate results in registers.

Order of Evaluation

1. Calculate $d + e$
2. Calculate $c - (d + e)$
3. Calculate $a + b$
4. Subtract the second result from the first result

Register Allocation

- R1 is used for $d + e$ and later $a + b$.
- R2 stores $c - (d + e)$.
- The final result is stored in R1.

Target Code

MOV R1, d

ADD R1, e

MOV R2, c

SUB R2, R1

MOV R1, a

ADD R1, b

SUB R1, R2

Simulation

MOV R1, d

R1 = d

ADD R1, e

R1 = $d + e$

MOV R2, c

R2 = c

SUB R2, R1

R2 = $c - (d + e)$

MOV R1, a

ADD R1, b

R1 = $a + b$

Finally:

SUB R1, R2

Therefore:

$$R1 = (a + b) - (c - (d + e))$$

Thus, the final result is available in R1.

(b) Target Code Using One Register

When only one register R1 is available, intermediate results cannot all be kept in registers. Therefore, temporary memory locations such as TEMP1, TEMP2, and TEMP3 are used.

Handling Intermediate Results

First calculate:

$$TEMP1 = d + e$$

Then:

$$TEMP2 = c - TEMP1$$

Next:

$$TEMP3 = a + b$$

Finally:

$$RESULT = TEMP3 - TEMP2$$

Target Code

MOV R1, d

ADD R1, e

MOV TEMP1, R1

MOV R1, c

SUB R1, TEMP1

MOV TEMP2, R1

MOV R1, a

ADD R1, b

MOV TEMP3, R1

MOV R1, TEMP3

SUB R1, TEMP2

MOV RESULT, R1

Simulation

Initially:

$$R1 = d$$

After ADD R1, e:

$$R1 = d + e$$

Store it:

$$TEMP1 = d + e$$

Next:

$$R1 = c$$

$$R1 = c - TEMP1$$

Therefore:

$$TEMP2 = c - (d + e)$$

Next:

R1 = a

R1 = a + b

Therefore:

TEMP3 = a + b

Finally:

R1 = TEMP3

R1 = TEMP3 - TEMP2

Hence:

RESULT = (a + b) - (c - (d + e))

Comparison

Feature	Two Registers	One Register
Registers used	R1, R2	R1
Temporary memory	Less	More
Intermediate values	Mostly in registers	Stored in memory
MOV instructions	Fewer	More
Code efficiency	Higher	Lower

Conclusion: Using two registers produces more efficient target code because intermediate results can remain in registers. With only one register, temporary memory locations are required to preserve intermediate results, resulting in additional memory operations.

Output:

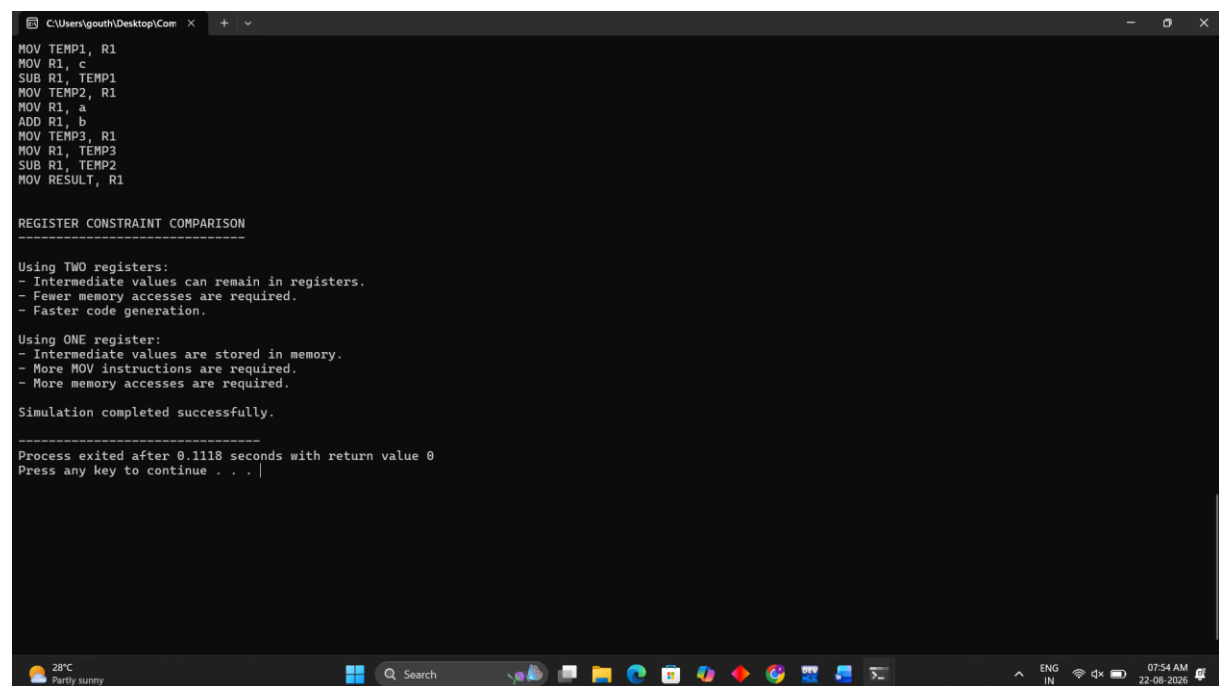
```
C:\Users\gouth\Desktop\Com x + v
MOV TEMP1, R1
MOV R1, c
SUB R1, TEMP1
MOV TEMP2, R1
MOV R1, a
ADD R1, b
MOV TEMP3, R1
MOV R1, TEMP3
SUB R1, TEMP2
MOV RESULT, R1

REGISTER CONSTRAINT COMPARISON
-----
Using TWO registers:
- Intermediate values can remain in registers.
- Fewer memory accesses are required.
- Faster code generation.

Using ONE register:
- Intermediate values are stored in memory.
- More MOV instructions are required.
- More memory accesses are required.

Simulation completed successfully.

-----
Process exited after 0.1118 seconds with return value 0
Press any key to continue . . .
```



Q5. Code Optimization Simulation

Given C++ code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {  
    for (int i = 0; i < n; i++) {  
        a[i] = x * y + z;  
        a[i + n] = x * y + w;  
    }  
}
```

(a) Common Subexpression Elimination and Code Motion

Step 1: Identify the common sub-expression

Inside the loop, we have:

$a[i] = x * y + z;$

$a[i+n] = x * y + w;$

The expression:

$x * y$

occurs **twice in every loop iteration**.

Therefore, $x * y$ is a **common sub-expression**.

Also, x , y , z , and w do not change inside the loop. Therefore, $x * y$ is **loop invariant**.

Step 2: Original computation

For every iteration, the compiler performs:

$t1 = x * y$

$t2 = t1 + z$

$a[i] = t2$

$t3 = x * y$

$t4 = t3 + w$

$a[i+n] = t4$

Here, $x * y$ is calculated twice.

For n iterations:

Number of $x*y$ computations = $2n$

Step 3: Apply Common Subexpression Elimination

Calculate $x * y$ only once:

$t = x * y$

for (...)

{

$a[i] = t + z;$

$a[i+n] = t + w;$

}

Step 4: Apply Code Motion

Since $x * y$ does not depend on i , move it outside the loop.

Optimized code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {
```

```
    int t = x * y;
```

```
    for (int i = 0; i < n; i++) {
```

```
        a[i] = t + z;
```

```
        a[i + n] = t + w;
```

```
    }
```

```
}
```

TAC Representation

Original:

L1:

```
    if i >= n goto L2
```

```
    t1 = x * y
```

```
    t2 = t1 + z
```

```
    a[i] = t2
```

```
    t3 = x * y
```

```
    t4 = t3 + w
```

```
    a[i+n] = t4
```

```
    i = i + 1
```

```
    goto L1
```

L2:

Optimized:

```
t1 = x * y
```

L1:

```
    if i >= n goto L2
```

```
    t2 = t1 + z
```

```
    a[i] = t2
```

```
    t3 = t1 + w
```

```
    a[i+n] = t3
```

```
    i = i + 1
```

```
    goto L1
```

L2:

(b) Analysis of Optimized Code

(i) Reduction in Number of Computations

Before optimization

For each iteration:

$x * y$

$x * y$

Therefore:

2 multiplications per iteration

For n iterations:

$2n$ multiplications

After optimization

The multiplication is performed once before entering the loop:

$t = x * y$

Therefore:

1 multiplication total

The number of multiplications is reduced from:

$2n \rightarrow 1$

This is a significant improvement when n is large.

Addition operations

The additions are still required:

$t + z$

$t + w$

So there are:

2 additions per iteration

or:

$2n$ additions

The main optimization is the elimination of repeated multiplication.

Benefits of Optimization

1. **Common subexpression elimination:** The repeated $x * y$ computation is performed only once.
2. **Loop-invariant code motion:** $x * y$ is moved outside the loop because its value does not depend on i .
3. **Reduced execution time:** Fewer multiplication operations are performed.
4. **Better performance for large n :** The saving becomes larger as the number of loop iterations increases.
5. **Efficient generated code:** The compiler can reuse the calculated value instead of recomputing it.

Final Comparison

Operation	Original Code	Optimized Code
$x * y$	$2n$ times	1 time
$+ z$	n times	n times

Operation Original Code Optimized Code

+ w n times n times

Total multiplications 2n 1

Conclusion: By applying **common subexpression elimination** and **loop-invariant code motion**, the repeated computation $x * y$ is calculated only once and reused throughout the loop. This reduces computational overhead and improves the execution efficiency of the program.

Output:

```
C:\Users\gouth\Desktop\Com x + v
ORIGINAL CODE SIMULATION
Iteration 0: x*y calculated twice, a[i] = 25, a[i+n] = 28
Iteration 1: x*y calculated twice, a[i] = 25, a[i+n] = 28
Iteration 2: x*y calculated twice, a[i] = 25, a[i+n] = 28
Iteration 3: x*y calculated twice, a[i] = 25, a[i+n] = 28
Iteration 4: x*y calculated twice, a[i] = 25, a[i+n] = 28

OPTIMIZED CODE SIMULATION
x * y calculated once = 28
Iteration 0: reused x*y, a[i] = 25, a[i+n] = 28
Iteration 1: reused x*y, a[i] = 25, a[i+n] = 28
Iteration 2: reused x*y, a[i] = 25, a[i+n] = 28
Iteration 3: reused x*y, a[i] = 25, a[i+n] = 28
Iteration 4: reused x*y, a[i] = 25, a[i+n] = 28

BENEFITS OF OPTIMIZATION
1. Common sub-expression x*y is calculated only once.
2. Loop-invariant computation is moved outside the loop.
3. Number of multiplications is reduced.
4. Execution time is reduced.
5. Computed value is reused in every iteration.

Simulation completed successfully.

-----
Process exited after 0.2975 seconds with return value 0
Press any key to continue . . .
```